

Project 2

Teresa Macarulla Uriarte
(Dated: October 2, 2025)

<https://github.uio.no/teresm/Comp.Phy.Project/tree/master/P2>

PROBLEM 1

Starting from

$$\gamma \frac{d^2 u(x)}{dx^2} = -F u(x), \quad (1)$$

introduce the dimensionless variable

$$\hat{x} \equiv \frac{x}{L} \in [0, 1] \quad \Longleftrightarrow \quad x = L\hat{x}.$$

By the chain rule,

$$\frac{d}{dx} = \frac{d\hat{x}}{dx} \frac{d}{d\hat{x}} = \frac{1}{L} \frac{d}{d\hat{x}}, \quad (2)$$

and hence

$$\frac{d^2}{dx^2} = \left(\frac{1}{L} \frac{d}{d\hat{x}} \right)^2 = \frac{1}{L^2} \frac{d^2}{d\hat{x}^2}. \quad (3)$$

Abusing notation slightly, denote by $u(\hat{x})$ the original displacement expressed as a function of $\hat{x}$. Substituting (3) into (1) gives

$$\gamma \frac{1}{L^2} \frac{d^2 u(\hat{x})}{d\hat{x}^2} = -F u(\hat{x}). \quad (4)$$

Multiplying by L^2/γ and defining $\lambda \equiv \frac{FL^2}{\gamma}$, we obtain the scaled equation

$$\frac{d^2 u(\hat{x})}{d\hat{x}^2} = -\lambda u(\hat{x}), \quad \hat{x} \in [0, 1], \quad (5)$$

PROBLEM 2

We now consider the discretized eigenvalue problem

$$A\vec{v} = \lambda\vec{v}, \quad (6)$$

where $A = \text{tridiag}(a, d, a)$ is a tridiagonal matrix. For $N = 6$ (with $h = 1/n$, $n = N + 1 = 7$, hence $a = -1/h^2 = -49$, $d = 2/h^2 = 98$), the C++ program implemented in the file `problem2.cpp` (available in the GitHub repository) constructs the matrix A , solves the eigenvalue problem numerically, and compares the results to the analytical eigenpairs.

The analytical eigenvalues and eigenvectors are given by

$$\lambda^{(j)} = d + 2a \cos\left(\frac{j\pi}{N+1}\right), \quad j = 1, \dots, N, \quad (7)$$

and

$$v_k^{(j)} = \sin\left(\frac{jk\pi}{N+1}\right), \quad j, k = 1, \dots, N, \quad (8)$$

The eigenvalues and eigenvectors computed with Armadillo and those obtained analytically from Eqs. (7) and (8) have been written to the file `eval_evec.txt`. Table I reports the values for the case $N = 6$. One observes that the numerical and analytical results coincide to machine precision for both eigenvalues and eigenvectors (once the latter are normalised and aligned up to an overall sign).

TABLE I: Eigenvalues and eigenvectors for $N = 6$. Scientific notation with 14 digits is used, matching the contents of `eval_evec.txt`.

Eigenvalues						
Analytical	9.70505094556293e+00	3.68979994178441e+01	7.61929484722812e+01	1.19807051527719e+02	1.59102000582156e+02	1.86294949054437e+02
Numerical	9.70505094556293e+00	3.68979994178441e+01	7.61929484722812e+01	1.19807051527719e+02	1.59102000582156e+02	1.86294949054437e+02
Analytical eigenvectors (6×6)						
2.31920613924330e-01	4.17906505941275e-01	5.21120889169602e-01	5.21120889169602e-01	4.17906505941275e-01	2.31920613924330e-01	
4.17906505941275e-01	5.21120889169602e-01	2.31920613924330e-01	-2.31920613924330e-01	-5.21120889169602e-01	-4.17906505941275e-01	
5.21120889169602e-01	2.31920613924330e-01	-4.17906505941275e-01	-4.17906505941275e-01	2.31920613924330e-01	5.21120889169602e-01	
5.21120889169602e-01	-2.31920613924330e-01	-4.17906505941275e-01	4.17906505941275e-01	2.31920613924330e-01	-5.21120889169602e-01	
4.17906505941275e-01	-5.21120889169602e-01	2.31920613924330e-01	2.31920613924330e-01	-5.21120889169602e-01	4.17906505941275e-01	
2.31920613924330e-01	-4.17906505941275e-01	5.21120889169602e-01	-5.21120889169602e-01	4.17906505941275e-01	-2.31920613924330e-01	
Numerical eigenvectors (6×6)						
2.31920613924330e-01	-4.17906505941275e-01	5.21120889169602e-01	5.21120889169603e-01	4.17906505941275e-01	-2.31920613924329e-01	
4.17906505941275e-01	-5.21120889169602e-01	2.31920613924330e-01	-2.31920613924330e-01	-5.21120889169602e-01	4.17906505941275e-01	
5.21120889169602e-01	-2.31920613924330e-01	-4.17906505941275e-01	-4.17906505941275e-01	2.31920613924329e-01	-5.21120889169602e-01	
5.21120889169602e-01	2.31920613924330e-01	-4.17906505941275e-01	4.17906505941275e-01	2.31920613924330e-01	5.21120889169602e-01	
4.17906505941275e-01	5.21120889169602e-01	2.31920613924330e-01	2.31920613924330e-01	-5.21120889169602e-01	-4.17906505941275e-01	
2.31920613924330e-01	4.17906505941275e-01	5.21120889169602e-01	-5.21120889169602e-01	4.17906505941275e-01	2.31920613924330e-01	

PROBLEM 3

Problem a

In order to apply the Jacobi algorithm to a matrix A and solve Eq 6, it is necessary to determine the largest (in absolute value) off-diagonal element. The function `max_offdiag_symmetric.cpp`, available in the GitHub repository, takes as input a symmetric matrix, identifies the element with the largest absolute value, and returns this value together with its position in the matrix.

Problem b

To verify that the function `max_offdiag_symmetric.cpp` works correctly, it has been applied to the matrix

$$A = \begin{bmatrix} 1 & 0 & 0 & 0.5 \\ 0 & 1 & -0.7 & 0 \\ 0 & -0.7 & 1 & 0 \\ 0.5 & 0 & 0 & 1 \end{bmatrix}.$$

For this small test, a C++ program called `problem3` was implemented, which defines the matrix and applies the function through the command `max_val=max_offdiag_symmetric(A,k,l)`. The result obtained is as expected: the maximum absolute off-diagonal value is 0.7, located at the positions $(i,j) = (2,3)$ or $(3,2)$.

It should be noted that the indexing of matrices (i.e., the position of a given element within a matrix) is handled differently in Armadillo. In general mathematical notation, the first element of a matrix is associated with the indices $(i,j) = (1,1)$, whereas in Armadillo the same element corresponds to $(i,j) = (0,0)$. This difference arises because both row and column indices in Armadillo start from zero instead of one. Consequently, the values k and l returned by the function, which represent the position of the element with the largest absolute value, must be incremented by +1 in order to match the conventional mathematical indexing.

PROBLEM 4

Problem a

The following algorithms presents the procedure to implement Jacobi's rotation and to solve Eq 6.

Algorithm 1 Jacobi eigenvalue algorithm for a real symmetric matrix

Input: symmetric $A \in \mathbb{R}^{N \times N}$, tolerance $\varepsilon > 0$, maximum iterations $K_{\max}$
Output: eigenvalues $\lambda_1, \dots, \lambda_N$ (on $\text{diag}(A)$), eigenvectors in $R \in \mathbb{R}^{N \times N}$ (columns)
 $R \leftarrow I_N$ $\triangleright$ accumulates the orthogonal similarity transforms
 $k \leftarrow 0$
while $\max_{i < j} |A_{ij}| > \varepsilon$ **and** $k < K_{\max}$ **do**
 $(p, q) \leftarrow \text{MAXOFFDIAGSYMMETRIC}(A)$ $\triangleright$ return indices $p < q$ of the largest off-diagonal $|A_{pq}|$ (Alg. 3)
 $\text{JACOBIROTATE}(A, R, p, q)$ $\triangleright$ apply a single Jacobi rotation (Alg. 2)
 $k \leftarrow k + 1$
 $\lambda_i \leftarrow A_{ii}$ for $i = 1, \dots, N$ $\triangleright$ diagonal now holds eigenvalues
return $(\lambda_1, \dots, \lambda_N)$ and R

Algorithm 2 Single Jacobi rotation on (p, q)

Input/Output: symmetric A , eigenvector accumulator R , indices $p < q$

// compute c and s (cosine and sine of the rotation)
if $A_{pq} \neq 0$ **then**
 $\tau \leftarrow \frac{A_{qq} - A_{pp}}{2A_{pq}}$
 $t_1 \leftarrow -\tau + \sqrt{1 + \tau^2}$
 $t_2 \leftarrow -\tau - \sqrt{1 + \tau^2}$
 $t \leftarrow \min(t_1, t_2)$
 $c \leftarrow \frac{1}{\sqrt{1 + t^2}}, \quad s \leftarrow ct$
else
 $c \leftarrow 1, s \leftarrow 0$

// update rows/columns p and q of A
for $i = 1, \dots, N$ with $i \neq p, q$ **do**
 $a_{ip}^{\text{new}} \leftarrow c A_{ip} - s A_{iq}$
 $a_{iq}^{\text{new}} \leftarrow s A_{ip} + c A_{iq}$
 $A_{ip} \leftarrow a_{ip}^{\text{new}}, \quad A_{pi} \leftarrow a_{ip}^{\text{new}}$ $\triangleright$ keep symmetry
 $A_{iq} \leftarrow a_{iq}^{\text{new}}, \quad A_{qi} \leftarrow a_{iq}^{\text{new}}$
 $a_{pp}^{\text{new}} \leftarrow c^2 A_{pp} - 2cs A_{pq} + s^2 A_{qq}$
 $a_{qq}^{\text{new}} \leftarrow s^2 A_{pp} + 2cs A_{pq} + c^2 A_{qq}$
 $A_{pp} \leftarrow a_{pp}^{\text{new}}, \quad A_{qq} \leftarrow a_{qq}^{\text{new}}, \quad A_{pq} \leftarrow 0, \quad A_{qp} \leftarrow 0$

// update eigenvector matrix R
for $i = 1, \dots, N$ **do**
 $r_{ip}^{\text{new}} \leftarrow c R_{ip} - s R_{iq}$
 $r_{iq}^{\text{new}} \leftarrow s R_{ip} + c R_{iq}$
 $R_{ip} \leftarrow r_{ip}^{\text{new}}, \quad R_{iq} \leftarrow r_{iq}^{\text{new}}$

Algorithm 3 Index of largest off-diagonal (absolute) entry in a symmetric matrix

Input: symmetric $A \in \mathbb{R}^{N \times N}$
Output: indices (p, q) with $p < q$ such that $|A_{pq}| = \max_{i < j} |A_{ij}|$
 $(p, q) \leftarrow (1, 2); m \leftarrow |A_{12}|$
for $i = 1, \dots, N$ **do**
 for $j = i + 1, \dots, N$ **do**
 if $|A_{ij}| > m$ **then**
 $m \leftarrow |A_{ij}|, (p, q) \leftarrow (i, j)$
return (p, q)

Problem b

A C++ program called `problem4.cpp` has been developed to apply Jacobi's rotation method for solving the eigenvalue problem defined in Eq. 6. This program makes use of the functions `Jacobi_rotation.cpp` and

`max_offdiag_symmetric.cpp`, which implement the operations described in Algorithms 2 and 3, respectively. The program was applied to the case $N = 6$, which corresponds to the same problem described in Problem 2 (Section 2 of this document).

The results obtained by executing `problem4.cpp` are presented in Table II. By comparing the results presented in Tables I and II, it is observed that the eigenvalues and eigenvectors coincide. We therefore conclude that the Jacobi method has been successfully implemented.

TABLE II: Eigenvalues and eigenvectors obtained using Jacobi's rotation algorithm for the case $N = 6$.

Eigenvalues					
3.6898×10^1	1.5910×10^2	9.7051×10^0	1.1981×10^2	7.6193×10^1	1.8629×10^2
Eigenvectors (6×6)					
0.4179	-0.4179	-0.2319	0.5211	-0.5211	-0.2319
0.5211	0.5211	-0.4179	-0.2319	-0.2319	0.4179
0.2319	-0.2319	-0.5211	-0.4179	0.4179	-0.5211
-0.2319	-0.2319	-0.5211	0.4179	0.4179	0.5211
-0.5211	0.5211	-0.4179	0.2319	-0.2319	-0.4179
-0.4179	-0.4179	-0.2319	-0.5211	-0.5211	0.2319

PROBLEM 5
Problem a

A C++ program, `problem5.cpp`, was implemented as an extension of `problem4.cpp` by adding an outer loop over the matrix size N . Specifically, N is varied from 6 to 12. For each N ; the discretization count $n = N + 1$, the mesh spacing $h = 1/n$, and the tridiagonal entries $a = -1/h^2$ and $d = 2/h^2$ are recomputed, the Jacobi rotation method is applied to solve the eigenvalue problem from Eq 6 and the process is repeated until the matrix A is (numerically) diagonal.

For every N in this range, the program stores the computed eigenvalues and eigenvectors, together with the total number of Jacobi rotations (transformations) required to diagonalize A , in the file `eval_evec_trans.txt`. The resulting growth of the number of transformations with N is illustrated in Fig. 1.

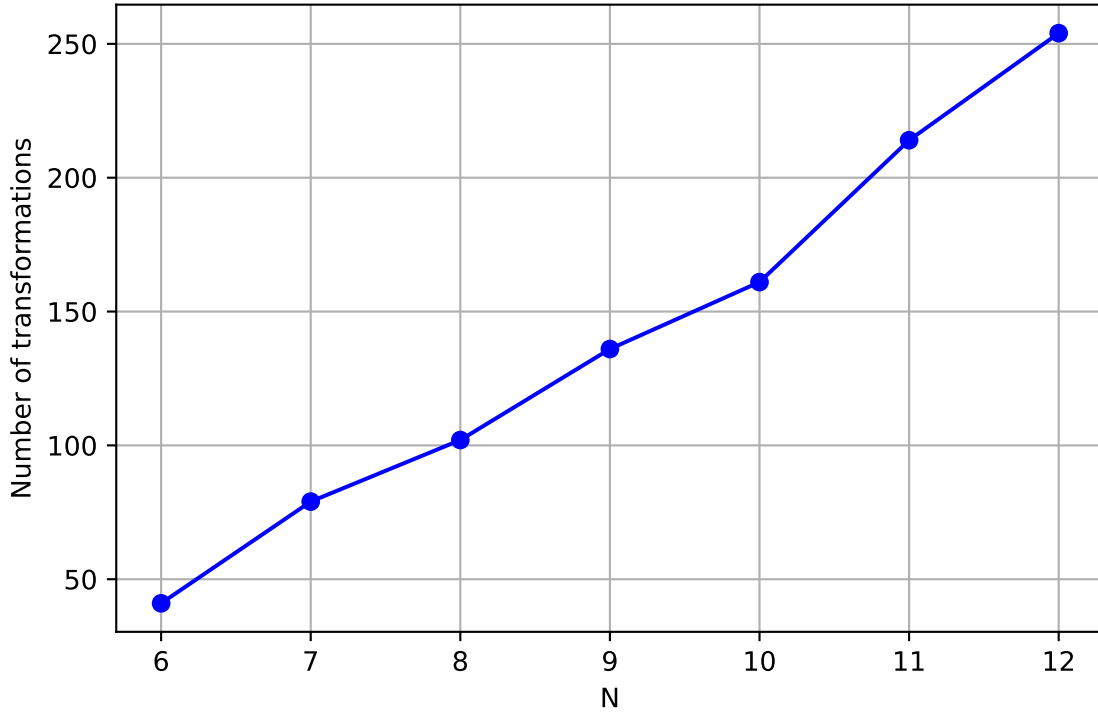


FIG. 1: Number of Jacobi rotations (transformations) required to diagonalize the tridiagonal matrix A as a function of matrix size N for $N = 6, \dots, 12$.

Problem b

For a dense $N \times N$ symmetric matrix, the number of off-diagonal entries is $\frac{N(N-1)}{2}$ (N^2 order). By contrast, for a tridiagonal matrix the number of off-diagonal entries is only $3N - 2$, so the observed transformation counts grow roughly linearly with N (See Fig 1). Moving from tridiagonal to dense therefore changes the expected scaling of the required Jacobi transformations from linear to quadratic growth.

PROBLEM 6

Problem a

For this final task, a C++ program named `problem6.cpp` was developed to compute the eigenvalues and eigenvectors, both through Jacobi's rotation algorithm and by means of analytical expressions (Eq. 8). The case of $n = 10$ steps was studied, and in both approaches the three lowest eigenvalues together with their corresponding eigenvectors were identified (see Eq. 6).

Subsequently, a Python script, `eval_vs_x.py`, was implemented to collect the values of these identified eigenvectors and plot them against the discretized variable x . Figure 2 displays the components of the different eigenvectors (three for each case) as a function of the discretized variable x .

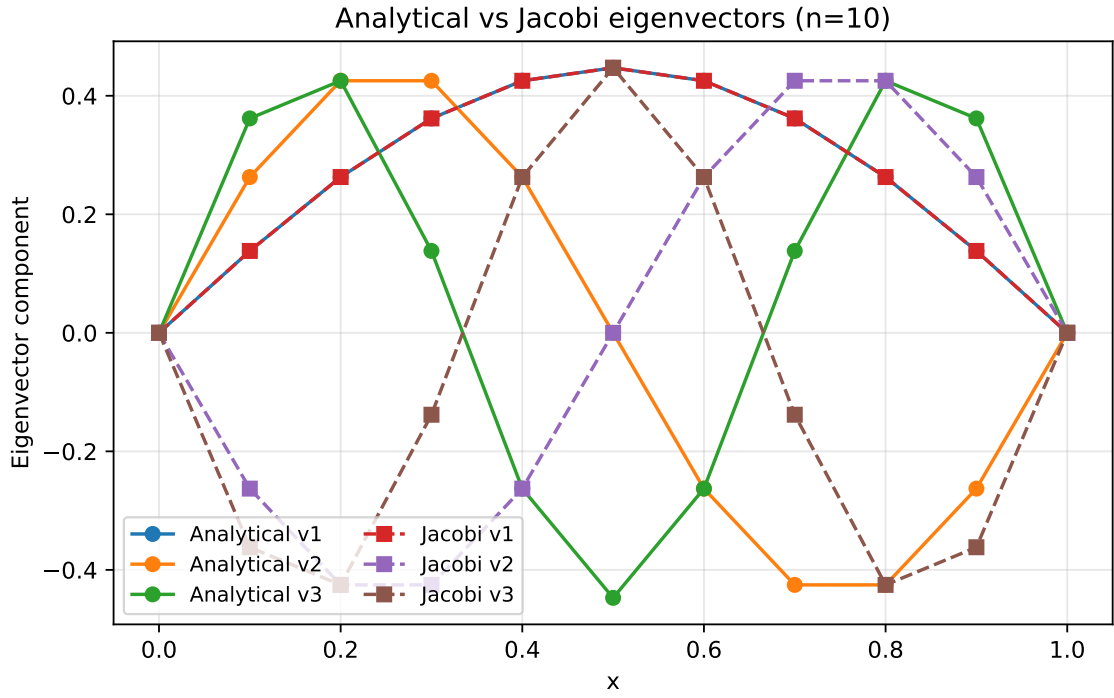


FIG. 2: Analytical (circles) and Jacobi (rectangles) eigenvectors v_1-v_3 plotted against the discretized coordinate x for $n = 10$.

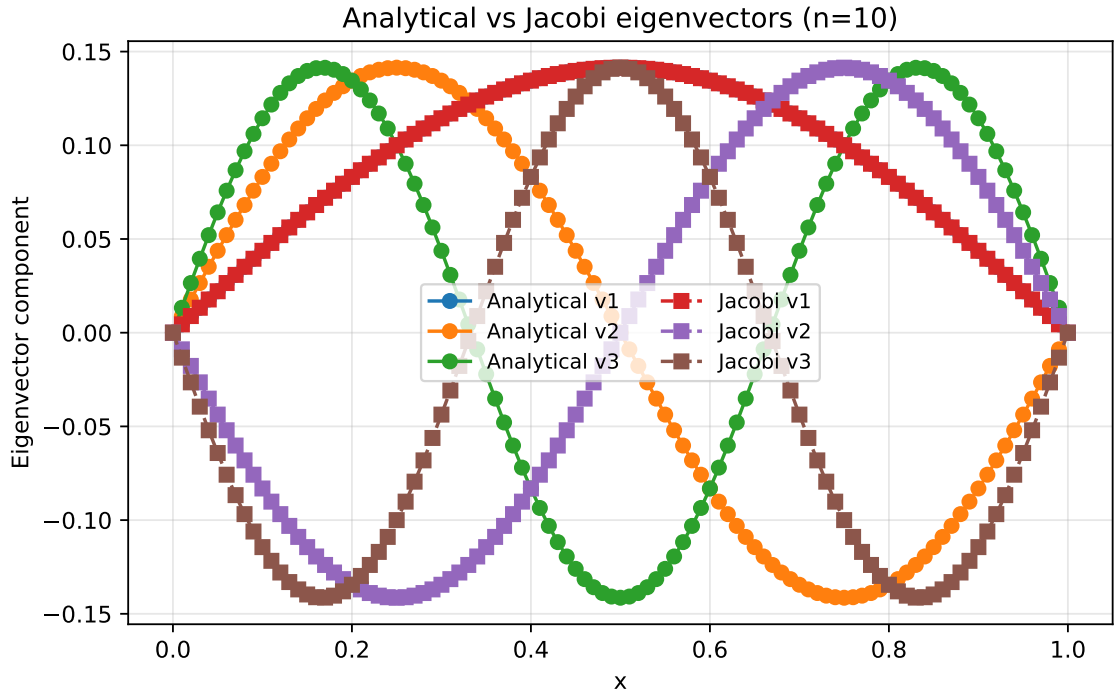


FIG. 3: Analytical (circles) and Jacobi (rectangles) eigenvectors v_1-v_3 plotted against the discretized coordinate x for $n = 100$.

Problem b

Analogously to the previous case, `problem6.cpp` tests the discretization with $n = 100$ and obtains the result shown in Figure 3 through `eval_vs_x.py`.

The figure confirms the behavior already observed for $n = 10$: one of the three eigenvectors matches almost pointwise between the analytical solution and the Jacobi method, while the remaining two appear with opposite signs. This is fully consistent with the theory of eigenproblems: each eigenvector is defined only up to a nonzero scalar multiple, so a global sign flip produces an equivalent eigenvector that spans the same one-dimensional eigenspace. In the present setting—“a special case of a one-dimensional buckling beam”—the eigenfunctions are mode shapes that can be taken as sinusoidal functions satisfying the boundary conditions; changing their overall sign yields the same physical mode. Hence, the observed sign differences do not indicate a discrepancy, but rather the inherent sign indeterminacy of eigenvectors. The close agreement of the magnitudes and spatial profiles across all three modes further validates the correctness of both the analytical construction and the Jacobi implementation at $n = 100$.